PDF עברי ל-Markdown — אפיון גרסה ראשונה

הבעיה

מי שממיר PDF בעברית לטקסט או ל-Markdown מקבל לעיתים קרובות פלט שבור: מילים הפוכות ("תוריכמם תוסנכה" במקום "הכנסות ממכירות"), אנגלית הפוכה בתוך עברית ("noriB rimA"), פיסוק שקופץ לתחילת השורה, ומספרים שזזים בין עמודות. זה קורה ב-Unstructured, ב-pdf_oxide, ב-poppler, ב-PyMuPDF וב-pypdfium2, כל אחד בצורה אחרת ובקבצים אחרים.

הבעיה האמיתית היא לא שהכלים נכשלים. הבעיה היא שהם נכשלים בשקט. כל הכלים שבדקנו מחזירים תוצאה תמיד, גם כשהיא הפוכה לגמרי, בלי שום סימן שמשהו לא בסדר. מי שמכניס את הפלט למערכת RAG או מדביק אותו לסוכן לא יודע שהוא עובד עם זבל.

המוצר: דף באתר, "PDF עברי ל-Markdown". מעלים קובץ, מקבלים Markdown נקי, ולצד כל עמוד שהכלי לא בטוח בו מופיעה אזהרה ברורה.

הריפו: `amirbiron/hebrew-pdf-to-markdown` (פרטי).

מצב נוכחי

סבב	מה נעשה	זהות מתוך 40 השורות המתויגות	שורות שנכשלו בבדיקת התצוגה (מתוך 3,541)
1	מערך בדיקה, ו-40 שורות מתויגות ומאושרות	11 (האב-טיפוס)	—
2	הקורא כחבילה, סדר לפי אלגוריתם הכיוון של Unicode, סוגריים	19	2 (היו 172)
3	ריווח לפי הפערים בעמוד, וכיוון לפי המסמך	26	2
4	גבולות בין תאים ותיבות, איחוד שורות טבלה שנשברו	35	1
5	בניית Markdown	—	—

בשורה אחת: ארבעת הסבבים הראשונים פתרו את **הקריאה** ואת **הפריסה**: סדר עברית ואנגלית, סוגריים, ריווח, וגבולות בין תאים ותיבות. בקובץ הטבלאות 91.7% מהתאים יוצאים זהים. אף שורה שהייתה נכונה לא נשברה בדרך. מתוך 5 השורות שנשארו, 3 הן **תווים שלא קיימים בקובץ** (28, 29, 34). שורה 9 היא **תג בתוך עיגול** שהקורא עוד לא מזהה כרכיב נפרד. ושורה 32 **דו-משמעית**: כל הפערים בה שווים, ואין גבול שאפשר למדוד.

מה בדקנו

12 קובצי PDF אמיתיים בעברית, משמונה תוכנות יצירה שונות, 141 עמודים בסך הכול.

קובץ	תוכנה שיצרה אותו	עמודים
טופס אפיון לצ'טבוט	Google Docs	4
מסמך 6 עמודים	Google Docs	6
סקירת מוצר MentorOS	WeasyPrint	8
הצעת מחיר	WeasyPrint	7

קובץ	תוכנה שיצרה אותו	עמודים
מסמך הגדרות Social Publisher	WeasyPrint	2
שני מסמכים נוספים	WeasyPrint	2 + 4
ספר תדמית GPS	InDesign	16
פלאייר שיווקי	Illustrator	1
מצגת	Canva (דרך Ghostscript)	24
מסמך optimization	Chrome	1
דוח כספי רבעוני של רב בריח (מאיה)	Aspose	66

חמישה קוראים נבדקו: pdf_oxide (חילוץ טקסט ו-Markdown), poppler (`pdftotext`, רגיל ומצב פריסה), PyMuPDF, pypdfium2, וקורא ניסיוני שבונה שורות לפי מיקום האותיות על הדף.

קובץ בדיקה לטבלאות (מסבב 4): רפרנס משתני הסביבה של CodeBot, שהומר ל-PDF (63 עמודים לרוחב, 6 טבלאות, 468 שורות). המקור ידוע, ולכן דף התשובות נגזר ממנו אוטומטית, והשוואה נעשית תא מול תא בלי תיוג ידני. כל שורה בפלט מותאמת לשורה בדף התשובות לפי שם המשתנה בתא הראשון, ולא לפי מספר שורה. לפני סבב 4 רק 0.3% מהתאים יצאו זהים. אחרי סבב 4: 91.7% מהתאים ו-66% מהשורות.

ממצאים

1. הקבצים תקינים. הקוראים הם ששוברים

זה הממצא שמשנה את כל התוכנית. PyMuPDF ו-pypdfium2 מחזירים את האותיות בדיוק בסדר שבו נשמרו בקובץ, יחד עם המיקום של כל אחת. בדקנו בכל 12 הקבצים, בכל זוג אותיות עבריות סמוכות באותה שורה, אם הן שמורות מימין לשמאל (הסדר הנכון) או משמאל לימין (הפוך):

בכל 12 הקבצים, משמונה התוכנות, העברית שמורה בסדר הנכון. אפס זוגות בסדר הפוך.

כלומר, ההנחה ש"התוכנה שיצרה את ה-PDF שומרת עברית הפוכה" לא נכונה במדגם הזה. הכשלים שראינו נוצרים בקוראים עצמם, שמנסים "לתקן" סדר שלא היה צריך תיקון ומקלקלים אותו, כל אחד לפי היוריסטיקה אחרת.

> [!NOTE] pdf_oxide לא יכול לשמש לבדיקה הזו: הוא מחזיר את האותיות כבר ממוינות משמאל לימין בכל הקבצים, כך שאין ממנו גישה לסדר השמור.

2. אין קורא מוכן שעובד על הכול

כדי למדוד בלי לקרוא כל קובץ ידנית השתמשנו בהתחלה בכמה בדיקות עקיפות: אותיות סופיות (ך ם ן ף ץ מופיעות רק בסוף מילה, אז אחת מהן בתחילת מילה מסגירה מילה הפוכה), מילים באנגלית מול poppler (שהאנגלית שלו תמיד תקינה), ואחוז השורות העבריות שמתחילות בסימן פיסוק (".זה מנטור שזוכר" מסגיר שורה שהסדר בה שבור).

קורא	עברית	אנגלית	פיסוק בתחילת שורה
pdf_oxide טקסט	0%–100%, תלוי בקובץ	0%–100%	—
pdf_oxide Markdown	0%–94%, תלוי בקובץ	0%–100%	—
poppler	100% באותיות, אבל סדר המילים בשורה הפוך	100%	—
PyMuPDF / pypdfium2 טקסט רגיל	100% בכל הקבצים	100% בכל הקבצים	עד 64% מהשורות
בונה שורות לפי מיקום	**100% בכל הקבצים**	**100% בכל הקבצים**	**0% בכל הקבצים**

> [WARNING!] **100% בבדיקה עקיפה הוא לא הוכחה.** ראינו כבר פעם אחת איך poppler קיבל 100% כשסדר המילים אצלו היה הפוך. לכן מאז סבב 1 המדידה היא מול 40 שורות מתויגות ומאושרות, ומאז סבב 2 גם מול בדיקת התצוגה (סעיף 3).

3. בונה שורות לפי מיקום, עם בדיקה עצמית

לא סומכים על הסדר שהקורא מחזיר. לוקחים מ-pypdfium2 כל אות עם המיקום שלה וקו הבסיס שלה, ומקבצים אותיות לשורות לפי קו הבסיס.

הסדר בתוך השורה (מסבב 2): לפי אלגוריתם הכיוון של Unicode (UAX #9), ולא לפי ביטוי רגולרי. הגרסה הראשונה זיהתה רצף אנגלי רק כשבין המילים היה רווח רגיל, ולכן "FastAPI + Python 3.12", "Meta, Telegram, Google" וביטויים עם חץ יצאו הפוכים. במקום להוסיף עוד מקרים, הקורא נשען על תכונה שהופכת את השאלה לבדיקה:

> **פלט נכון הוא פלט שאם מציגים אותו בפסקה עברית, הוא נראה בדיוק כמו השורה בעמוד.**

את מה שנראה בעמוד כבר יש: האותיות של השורה ממוינות לפי מיקום. לכן הקורא בונה את הסדר, מציג אותו בעצמו, ובודק שיוצא בדיוק מה שבעמוד. הבדיקה רצה על כל 3,541 השורות בסט, לא רק על המתויגות, והתוצאה שלה מופיעה בדוח לכל עמוד. השורות שנכשלו בה ירדו מ-172 ל-2.

כיוון הפסקה (מסבב 3, שונה בסבב 4): כיוון אחד לכל המסמך, לפי **רוב השורות** שיש בהן עברית, ולא לפי רוב האותיות. בקובץ הטבלאות יש יותר אותיות לטיניות מעבריות בגלל שמות המשתנים הארוכים, אבל ב-88% מהשורות יש עברית. לפי הכלל הקודם הקובץ נקרא משמאל לימין, והתאים יצאו בסדר הפוך. ב-12 הקבצים האחרים שני הכללים נותנים אותה תוצאה. במסמך עברי כל שורה נקראת כחלק מפסקה עברית, גם שורה בלי עברית. הקורא ובדיקת התצוגה משתמשים באותו מקור לכיוון, כך שאין שני עותקים של הכלל שיכולים להיפרד.

ריווח (מסבב 3): לפי מה שמצויר בעמוד ולא לפי תווי הרווח שבקובץ. תווי הרווח יורדים, ורווח נכנס בכל פער שגדול מהפער הרגיל בין אותיות **באותה שורה**, ועוד חצי רוחב רווח. בכותרת עם ריווח אותיות הפער הרגיל רחב, ולכן נספרים רק הרווחים האמיתיים בין מילים.

סוגריים: הצורה של כל סוגר נקבעת ממה שמצויר בעמוד ולא מהתו שבקובץ, כי יש קבצים שבהם התו והציור לא תואמים.

ביצועים: 0.01 עד 0.04 שניות לעמוד לבניית השורות; ציור הסוגריים מוסיף כ-4 שניות לכל הסט.

באג שנמצא בדרך ושווה לזכור: בגרסה הראשונה קובצו אותיות לשורות לפי תחתית האות, ואותיות שיורדות מתחת לשורה (ק ר ן ף ץ) נפלו לשורה אחרת. המעבר לקו הבסיס של האות פתר את זה.

4. גבול מובנה: שורות ששני סדרים שלהן נראים זהה

בחלק מהשורות, שני סדרים לוגיים שונים מוצגים על המסך בדיוק אותו דבר. למשל "Onboarding 1" ו-"Onboarding 1". במקרה כזה העמוד עצמו לא מכריע, ואף כלי לא יכול להכריע ממנו. נבדקו שלושה רמזים נוספים (גודל הפער, סדר השמירה בקובץ, חלוקה לאובייקטים), ואף אחד מהם לא עקבי בין תוכנות.

מה הקורא עושה: משאיר רק קריאות שמוצגות כמו העמוד; בין אלה מכריעה טיפוגרפיה (" .1" ו-" :Database" הן קריאה הפוכה); ואם עדיין נשארות כמה, ברירת המחדל היא שמספר שייך לאנגלית שמשמאלו, **והשורה מסומנת דו-משמעית.** בסט יש כ-45 שורות כאלה. ברירת המחדל נכונה ל-"IAS 34", "QT-00011", "VIP 500" ו-"ב-Python 3.12", ושוברת את שורות 9 ו-32. **שורה 9:** הספרה 1 מצוירת בתוך עיגול, כלומר היא רכיב נפרד, ומסבב 4 היא מתויגת כשני תאים. הפער בינה לבין הטקסט קטן מכדי להיחשב גבול, ולכן היא תמשיך להיכשל עד שהקורא ישתמש בצורות שמצוירות סביב טקסט (עיגול, מסגרת) כראיה לגבול, כמו שהוא כבר משתמש בקווי המסגרת של טבלה. **שורה 32:** כל הפערים בה שווים, כך שאין גבול שאפשר למדוד.

5. טבלאות

בדוח הכספי (עמוד 35, דוח רווח והפסד: שבע עמודות, כותרות בשלוש שורות, מספרים שליליים בסוגריים וכוכביות):

קורא	מה יצא
pdf_oxide טקסט	עברית הפוכה, שורות שהודבקו, **וכוכבית שזזה למספר בעמודה הסמוכה**
poppler רגיל	כל מספר בשורה נפרדת
poppler פריסה	שורות ועמודות נשמרו, רק הסוגריים של מספרים שליליים התהפכו
בונה שורות לפי מיקום	מסבב 4: תאים נפרדים, שורות טבלה שנשברו מאוחדות, והעברית, הכוכביות וסדר העמודות נכונים

[!NOTE] **תיקון לגרסה קודמת של המסמך:** כאן נכתב בעבר שאצל בונה השורות "סדר העמודות התהפך והכוכביות נצמדו לצד הלא נכון". זה נבע מניסוי מוקדם עם PyMuPDF. עם pypdfium2 וקיבוץ לפי קו הבסיס, כשמציגים את השורה בפסקה עברית היא נראית בדיוק כמו בעמוד, כולל הכוכביות.

[!CAUTION] **הכוכבית שזזה היא סוג הכשל שהמוצר קיים בשבילו.** במקור הכוכבית (סימון "סווג מחדש") היא על 19,138, ובפלט של pdf_oxide היא על 19,677. הפלט נראה תקין לגמרי ופשוט שגוי.

איך הקורא בונה טבלה (מסבב 4):

- **גבול בין רכיבים בשני כללים, כי סף אחד לא מספיק.** פער גדול פי 10 לפחות מהרווח הרגיל בין מילים הוא גבול תמיד. פער קטן יותר הוא גבול רק כשהוא חלק מ"מסדרון": רצועה אנכית ריקה שחוזרת באותו מקום בשורות רצופות, כמו עמודה. סף אחד לא היה מבדיל בין פער של פי 7.3 בתוך תא (שורה 13) לבין גבול בין תאים של פי 9.6 (שורה 38).
- **שורת טבלה חדשה** מזוהה לפי קווי המסגרת, כשיש קו בין רוב השורות. בלי קווים, כמו בדוח הכספי, שורה חדשה מתחילה כשיש טקסט בעמודה הראשונה.
- **שורות שנשברו** מתאחדות לתא אחד, עם רווח ביניהן. בלי רווח רק אחרי מקף או לוכסן שצמודים למילה, או כשהשורה הקודמת היא מילה אחת שממלאת את כל רוחב העמודה (מילה ארוכה שנשברה באמצע).

בדוחות כספיים יש בדיקה מובנית בחינם: המספרים חייבים להסתדר. הכנסות (366,343) פחות עלות המכירות (282,128) שווה בדיוק לרווח הגולמי (84,215).

6. רשימת הכשלים

נפתרו בסבב 2: סדר עברית ואנגלית (כולל רשימות עם פסיקים, פלוס וחיצים), סוגריים בשורות עבריות.

נפתרו בסבב 3: ריווח (כותרות עם ריווח אותיות, "ומהא יתך", רווחים בתוך מילים ב-Aspose, רווח בצד הלא נכון של סוגר), ושורה בלי עברית במסמך עברי (הלוכסן בסוף כתובת).

נפתרו בסבב 4: גבולות בין תאים ותיבות (שורות 2, 8, 11, 21, 30, 31, 38–40), ושורות טבלה שנשברו לכמה שורות על הדף.

על כל הסט, 22,640 פערים שהיה בהם תו רווח בקובץ: 319 (1.4%) הורדו כי אין להם פער בעמוד, ו-252 רווחים (1.1%) נוספו בפערים בלי תו רווח. השיעור הגבוה ביותר הוא ב-Google Docs (כ-5.5% בכל כיוון), ושם גם נמצאו הרווחים בצד הלא נכון של סוגריים.

נשארו:

כשל	דוגמה	כיוון לתיקון	סבב
עמודות שמתערבבות שורה-שורה	בפלאייר, תיבות משלוש עמודות באותה שורה בעמוד	פילוח עמודות לפני קיבוץ שורות	פתוח
תג בתוך צורה מצוירת	הספרה 1 בעיגול (שורה 9)	הצורה שמצוירת סביב הטקסט כראיה לגבול	פתוח
סוגריים זוויתיים ומירכאות	token בסוגריים זוויתיים ומירכאות שיוצאים הפוכים בטבלאות	הרחבת קביעת הצורה מהציור, אם זה השורש	5

כשל	דוגמה	כיוון לתיקון	סבב
קטעי קוד מבודדים	6 שורות בקובץ הטבלאות: WeasyPrint מציגה קטע קוד כמבודד משאר הטקסט, ולכן שום סדר לוגי רגיל לא מוצג כמו העמוד	גבול מובנה; לסמן	6
טקסט שלא ממופה לתווים בקובץ	"GPS" בפלאייר יוצא כתווי בקרה (U+001D–U+001F); סימן השאלה ב"ומה איתך?" כ-U+001C; אימוג'י במצגת כ-"¹" ו-U+009A	אי אפשר לשחזר מהקובץ. לזהות ולסמן את העמוד (ראו "פסילה מיידית")	6
סוגריים שמצוירים באותה צורה	ב-Google Docs ("מאפס(₪3.60 :(DALL-E 3 API") עובר את בדיקת התצוגה אבל שגוי במשמעות	בדיקת זוגיות סוגריים בציון	6
עמוד שהוא תמונה	כריכת ספר GPS	OCR, מחוץ לגרסה הראשונה	—

הגישה: קורא אחד, שורות לפי מיקום, בדיקות שמסמנות

1. **קריאה:** מ-pypdfium2, כל אות עם המיקום וקו הבסיס שלה. כשל בקריאת המיקום לא עובר בשקט אלא נספר לכל עמוד.
2. **שורות:** קיבוץ אותיות לשורות לפי קו הבסיס.
3. **סדר:** לפי אלגוריתם הכיוון של Unicode, עם בדיקה עצמית שהפלט מוצג בדיוק כמו העמוד. כיוון הפסקה נקבע לפי המסמך.
4. **ריווח:** לפי הפערים בעמוד, ביחס לפער הרגיל באותה שורה.
5. **רכיבים וטבלאות:** גבולות בין תאים ותיבות לפי פער גדול או מסדרון חוזר, שורות טבלה לפי קווי מסגרת או העמודה הראשונה, ואיחוד שורות שנשברו.
6. **בניית Markdown:** כותרות, רשימות וטבלאות מתוך המבנה שנבנה.
7. **ציון וסימון:** עמוד שלא עבר את הבדיקות יוצא עם אזהרה וסיבה, ולא נמסר כאילו הוא תקין.

מה זה 1:1 כאן: אותו תוכן, באותו סדר, עם אותן כותרות וטבלאות. לא אותו מראה; פונטים, צבעים ומיקום על הדף בכל מקרה לא עוברים ל-Markdown.

הציון והסימון

עקרונות שאומצו מההתייעצות על ארכיטקטורה:

כמה צירים, לא מספר אחד. ציון אחד מערבב שאלות שונות (סדר, זהות תווים, שלמות), ואז אין דרך לדעת מה נכשל.

ציר	מה הוא שואל	ממה הוא נבנה
סבירות	האם הטקסט נראה כמו עברית ועברית-אנגלית תקינה	בדיקת התצוגה (מסבב 2), אותיות סופיות, מספרים הפוכים ("5202", "65.432,1"), זוגיות סוגריים, מילת יחס בסוף שורה ("של", "על"), ושורות שסומנו דו-משמעיות
שלמות	האם כל התוכן הגיע, בלי לבדוק סדר	השוואת אוסף התווים (בלי סדר) מול הטקסט הרגיל של pypdfium2: אם תווים נשרו או הוכפלו, האוספים לא יתאימו
מאמץ תיקון	כמה תיקונים נדרשו כדי להגיע לתוצאה	ספירת התיקונים שהופעלו על העמוד (כולל רווחים שהורדו ונוספו), וכשלים בקריאת מיקום

כלל הסימון: עמוד מסומן "לא בטוח" אם הסבירות נמוכה, או אם השלמות נכשלה, או אם מאמץ התיקון גבוה (גם כשהסבירות מושלמת: עמוד שתוקן הרבה הוא עמוד שהקורא נכשל בו, וזה צריך להיראות).

פסילה מיידית, בלי ציון: עמוד בלי טקסט (תמונה), טקסט מסובב, או שיעור גבוה של תווים לא ממופים. "לא ממופים" כולל לא רק U+FFFD ואזור השימוש הפרטי של Unicode, אלא גם **תווי בקרה** (U+0000–U+001F, U+007F–U+009F) בתוך טקסט. סבב 1 הראה שזו הצורה הנפוצה בפועל, ושבדיקה רק של U+FFFD הייתה מפספסת את כל המקרים בסט.

עמוד קצר הוא לא עמוד בטוח. במקום ממוצע, משתמשים בגבול התחתון של רווח בר-סמך (Wilson): 9 מתוך 9 מילים תקינות נותן בערך 0.70, ו-200 מתוך 200 נותן בערך 0.98. כך עמוד כותרת עם כמה מילים לא מקבל ביטחון שהוא לא הרוויח.

בלי AI כשופט בגרסה ראשונה. הבעיה נפתרת מהמיקומים בלי AI, ושליחת מסמכים של משתמשים לספק חיצוני בכלי ציבורי היא סיכון פרטיות. בגרסה 2 אפשר לשקול אותו כמפענח אחרון לעמודים שסומנו, מאחורי מתג מפורש, ורק כבחירה בין שתי אפשרויות לשורה בודדת, לא כציון.

כיול ומדידה

- **40 שורות מתויגות ומאושרות** (`testset/labels.md`). כל שינוי נמדד מולן: כמה זהות לחלוטין, ומה המרחק ברמת התו בשאר.
- **בדיקת סדר מכנית לתיוגים (בתוך מערך הבדיקה מסבב 4):** כל "טקסט נכון" מוצג באלגוריתם התצוגה של החבילה ומושווה לסדר האותיות בעמוד, כולל הכיוון של כל סוגר. אדם בודק רק תוכן, והמכונה בודקת סדר. 37 מתוך 40 עוברות; שלוש השורות שלא עוברות הן התווים שלא קיימים בקובץ.
- **מפריד בין תאים בתיוג:** התג `br`. הוא לא יכול להתנגש עם התו `|`, שבתיוג הוא תמיד התו עצמו. ב-GitHub כל תא מוצג בשורה משלו.
- **תיוג של תא אחד** נמדד גם מול כל תא בנפרד, כדי שקורא שמפריד נכון בין תיבות לא ייענש.
- **דף תשובות אוטומטי לטבלאות** מקובץ שהמקור שלו ידוע (קובץ משתני הסביבה).
- **שגיאות מכוונות.** לוקחים שורות תקינות ומשחיתים אותן בכל צורות הכשל שראינו, כדי לראות כמה כל בדיקה תופסת כל כשל, ולקבוע סף בלי לכוון אותו על המדגם.
- **השארת תוכנה אחת בחוץ.** מכיילים על שבע תוכנות ובודקים על השמינית. אם הסף משתנה מאוד בין התוכנות, זה מידע בפני עצמו.

תיחום גרסה ראשונה

בפנים:

- PDF דיגיטלי (שיש בו שכבת טקסט), בעברית ובעברית-אנגלית מעורבת.
- טקסט, כותרות, רשימות, וטבלאות פשוטות.
- פלט Markdown, עם אזהרה ברמת העמוד.

בחוץ, לגרסה הבאה:

- סריקות ו-OCR (המועמד הוא `paddleocr-hebrew` של Rivok Labs, שמחזיר עברית בסדר הנכון ובבדיקה שלהם עקף את Tesseract בפער גדול על טקסט מעורב).
- AI כמפענח אחרון לעמודים שסומנו.
- טבלאות מורכבות ודוחות כספיים, כולל בדיקת סכומים.

כלל התיחום: "מסומן כלא בטוח" היא תוצאה לגיטימית, לא באג שחייבים לסגור. בלי הכלל הזה הפרויקט נגרר למרדף אחרי מקרי קצה.

הרכיבים

רכיב	מה הוא	הערות
pypdfium2	הקורא היחיד: אותיות, מיקומים, קו בסיס	התלות היחידה של הקורא בזמן ריצה. רישיון BSD / Apache 2.0. קו הבסיס דרך `FPDFText_GetCharOrigin`

רכיב	מה הוא	הערות
החבילה `hebrew_pdf`	בונה השורות, הסדר, הריווח והבדיקה העצמית	כולל מימוש משלה לאלגוריתם התצוגה, עם טסט שמשווה אותו ל-python-bidi
python-bidi	אלגוריתם תצוגה, לבדיקות בלבד	תלות פיתוח (`requirements-dev.txt`), רישיון LGPL-3.0. לא בקוד שרץ בשרת
מערך הבדיקה `harness`	מריץ את הקורא על הסט ומפיק `testset/report.md`	הדוח יציב בין הרצות, כך שה-diff שלו ב-PR הוא ההשוואה. כולל טבלת ריווח לפי תוכנה, מדידה תא מול תא בקובץ הטבלאות, ובדיקת סדר לתיוגים
הדף באתר	העלאה, תצוגה מקדימה, העתקה והורדה, אזהרה לצד כל עמוד שסומן עם הסיבה	

פרטיות: אנשים יעלו חשבוניות, חוזים ותלושים. הקובץ מעובד בזיכרון ונמחק מיד, בלי לשמור אותו ובלי לוגים של תוכן. כדאי לכתוב את זה בדף עצמו, כי זה מה שיקבע אם אנשים יעלו קבצים אמיתיים.

סדר הבנייה

1. ✅ **סבב 1:** מערך בדיקה ו-40 שורות מתויגות.
2. ✅ **סבב 2:** הקורא כחבילה, סדר לפי UAX #9 עם בדיקה עצמית, סוגריים. 19 מתוך 40.
3. ✅ **סבב 3:** ריווח לפי פערים בעמוד, וכיוון לפי המסמך. 26 מתוך 40.
4. ✅ **סבב 4:** גבולות בין תאים ותיבות, איחוד שורות טבלה, מדידה תא מול תא ובדיקת סדר לתיוגים. 35 מתוך 40, ו-91.7% מהתאים בקובץ הטבלאות.
5. **סבב 5, בשני PR-ים:**
   - **סגירת קצוות מסבב 4:** שורה 9 כשני תאים, הסרת תווים בלתי נראים מדף התשובות של הטבלאות, בדיקת רגישות לספים, ספירת חיבורים בלי רווח, וסוגריים זוויתיים ומירכאות.
   - **בניית Markdown:** פסקאות, כותרות, רשימות, טבלאות (כולל טבלה שנמשכת על כמה עמודים), הורדת כותרות עליונות ותחתונות, וגבולות עמודים. נמדד מול המקור של קובץ הטבלאות, בקריאה חוזרת של ה-Markdown במפענח אמיתי, ובבדיקת שלמות על כל הקבצים.
6. **סבב 6:** הציון, הסימון והכיול, כולל פסילה מיידית על תווים לא ממופים ובדיקת זוגיות סוגריים.
7. **הדף באתר.**

החלטות פתוחות

- **איפה הדף יושב:** בתוך CodeKeeper (לצד רעיון עמוד עורך ה-Markdown העברי לקידום אורגני) או שירות נפרד.
- **ציון הסף:** נקבע מהכיול, לא מראש.
- **מגבלות:** גודל קובץ ומספר עמודים מקסימלי לדף ציבורי.
- **תו כיוון בתחילת פסקה:** רוב המציגים קובעים את כיוון הפסקה לפי האות החזקה הראשונה בה, ולכן פסקה עברית שמתחילה באנגלית תוצג משמאל לימין. תו כיוון בלתי נראה מתקן את התצוגה, אבל מזיק למי שמעתיק את הטקסט או מכניס אותו ל-RAG. ההחלטה תתקבל אחרי הספירה בסבב 5.

מה עוד לא נבדק

- **מדגם התיוג קטן:** 40 שורות הן אמת, אבל לא הרבה. קובץ הטבלאות יוסיף 468 שורות עם אמת אוטומטית.
- **סריקות** בכלל.
- **טבלאות מורכבות** מעבר לעמוד אחד מדוח אחד.

- **מסמך שרובו באנגלית:** אין כזה בסט. כרגע, במסמך כזה שורה עברית עדיין נקראת כעברית (נבדק בטסט), אבל לא נבדק על קובץ אמיתי.
- **מדגם קבצים גדול יותר:** השאלה שמכריעה אם זה פרויקט של חודש או בור בלי תחתית היא אם רשימת הכשלים נשארת קצרה כשמוסיפים עוד 20-30 קבצים. עד עכשיו כל סבב קיצר אותה.
- **הספים של סבב 4:** אם התוצאה נשארת אותה תוצאה כשמזיזים את "פי 10" ואת ספי המסדרון. נבדק בסבב 5.
- **חיבור שורות בלי רווח בתוך תא:** הכלל נשען על כך שבקובץ הטבלאות מילים ארוכות נשברות באמצע, וזה נובע מהאופן שבו בנינו את הקובץ. בטבלאות עבריות אמיתיות כמעט אין שבירה כזו. מסבב 5 נספר כמה פעמים הכלל מופעל בשאר הקבצים.
- **משמעות LGPL** של python-bidi בשירות לא נבדקה משפטית; כרגע היא לא רצה בשרת, אז זה לא חוסם.

החלטות שהתקבלו בדרך

- **מכמה קוראים לקורא אחד.** התוכנית המקורית הייתה להריץ כמה קוראים על כל עמוד ולבחור את הטוב. אחרי שהתברר שהקבצים עצמם תקינים, קורא אחד שבונה שורות לפי מיקום עקף את כולם, והבחירה בין קוראים ירדה.
- **סדר לפי UAX #9 ולא לפי ביטוי רגולרי (סבב 2).** הביטוי היה טלאי שנשבר בכל צורה חדשה של רשימה באנגלית. האלגוריתם, יחד עם הבדיקה העצמית, הופך כל שורה בסט לבדיקה.
- **ברירת מחדל לדו-משמעות:** מספר שייך לאנגלית שמשמאלו, והשורה מסומנת. נכון ברוב המקרים בסט; החריגים הם רכיבים נפרדים.
- **שורה בלי עברית במסמך עברי — כיוון לפי המסמך (סבב 3).** גם שורה כזו נחשבת חלק מפסקה עברית. זה תיקן את הלוכסן שקפץ בכתובת, ולא שינה מילה אחת באנגלית (כמו "OSMentor"), שנראית זהה בשני הכיוונים.
- **רווחים לפי הפערים בעמוד ולא לפי תווי הרווח (סבב 3).** "הפער הרגיל" נמדד רק בין שתי אותיות, לא בין ספרות או פיסוק (אחרת מספרי עמודים כמו "2 / 8" נדבקו). רוחב רווח מוערך כחמישית מגובה האותיות, והתוצאה על כל הסט זהה בטווח רחב סביב ההערכה. הרוחב לא נלקח מהגופן, כי כשבגופן אין תו רווח, ספריית הקריאה מחזירה רוחב מופרך בלי אזהרה, ובקובץ של Chrome זה היה מוחק את כל הרווחים.
- **בלי חריגים לפי תוכנה.** כל כלל הוא יחסי לשורה או למסמך. כלל שצריך חריג לתוכנה מסוימת הוא התאמה למדגם, ולא נכנס.
- **כיוון המסמך לפי רוב השורות ולא לפי רוב האותיות (סבב 4).** שמות משתנים ארוכים הפכו מסמך עברי ל"אנגלי" בכלל הקודם.
- **גבול בין רכיבים בשני כללים (סבב 4):** פער גדול מאוד, או פער קטן יותר שחוזר באותו מקום כמו עמודה. סף אחד היה כוונן למדגם.
- **דף התשובות משקף את העמוד, לא את יכולת הקורא (סבב 4).** שורה 9 תויגה כשני תאים כי הספרה בעיגול היא רכיב נפרד, גם שזה אומר שהשורה תמשיך להיכשל.
- **בלי poppler:** היה הטוב ביותר לטבלאות, אבל הוא תוכנת מערכת (דורש פריסה עם Docker ב-Render) ברישיון GPL. נתיב הטבלאות ייבנה לפי מיקום במקומו.
- **בלי PyMuPDF:** עובד מצוין, אבל ברישיון AGPL, שבשירות אינטרנט ציבורי מחייב לפרסם את קוד השירות או לקנות רישיון מסחרי. pypdfium2 נותן את אותו מידע ברישיון פתוח.
- **בלי pdf_oxide:** מחזיר אותיות כבר ממוינות, ומצבי הטקסט וה-Markdown שלו נכשלים בקבצים שונים.
- **בלי מיזוג ברמת מילה:** מייצר טקסט שנראה תקין ולא ניתן לאימות.

מקורות

- ההתייעצות על מדידה ושילוב

> תקרא את זה עם הmcp של Mdocs, ממוקם בפרויקט "רעיונות לפיתוח"

- דוח מערך הבדיקה (עותק, אחרי סבב 3) - הקובץ נמחק ונמצא בריפו
- pypdfium2 ב-PyPI

- PyMuPDF — מבנה הפלט rawdict
- pdf_oxide — גלאי עברית בסדר חזותי (גרסה 0.3.54)
- paddleocr-hebrew (Rivok Labs)
- doc2md — עטיפה של MarkItDown ו-zerox, נבדק כהשראה ונפסל כבסיס